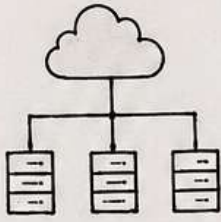


≡ SYSTEM DESIGN ≡ INTERVIEW HANDBOOK



Build Scalable, reliable & efficient systems

By : abhi_techhub

IG : abhi_techhub

What is System Design?

System design is the process of designing scalable, reliable, high-performance and cost-efficient systems to solve real-world problems.

Key Principles

- Scalability
- Reliability
- Availability
- Performance
- Consistency
- Cost Efficiency

Components in System Design

- Load Balancer
- Caching
- Database (Sharding & Replication)
- Message Queue (Kafka)
- In-Memory Store (Redis)
- API Gateway
- Rate Limiter
- URL Shortener Design



Think
scalable,
design
smart!

Overview of Topics

- | | |
|------------------------|---|
| ① Scalability | → How to handle growth and scale systems. |
| ② Load Balancer | → Distribute traffic across servers. |
| ③ Caching | → Improve performance by storing data. |
| ④ Database Sharding | → Split database horizontally for scale. |
| ⑤ Replication | → Keep data copies for high availability. |
| ⑥ Kafka | → Distributed message queue for streams. |
| ⑦ Redis | → In-memory data store for speed. |
| ⑧ API Gateway | → Single entry point for all APIs. |
| ⑨ Rate Limiter | → Control excessive requests. |
| ⑩ URL Shortener Design | → Design TinyURL-like system. |

Good system design is not about perfect code,
it's about solving the right problems at scale.



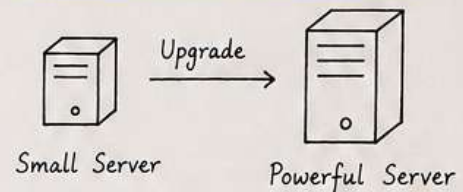
①. SCALABILITY

Scalability is the ability of a system to handle growing amounts of work (users, data, requests) by adding resources.

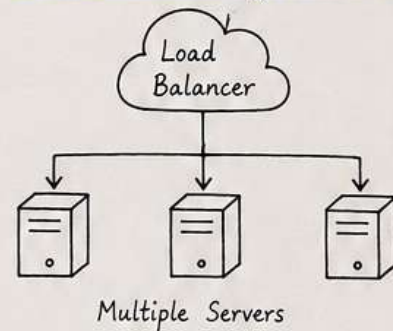
Types of Scalability

- Vertical Scaling (Scale Up)
 - Add more power (CPU, RAM, Disk) to a single machine.
- Horizontal Scaling (Scale Out)
 - Add more machines/instances behind a load balancer.

Vertical Scaling (Scale Up)



Horizontal Scaling (Scale Out)



Why Scalability?

- Handle traffic spikes
- Improve performance
- Ensure high availability
- Support business growth
- Optimize cost

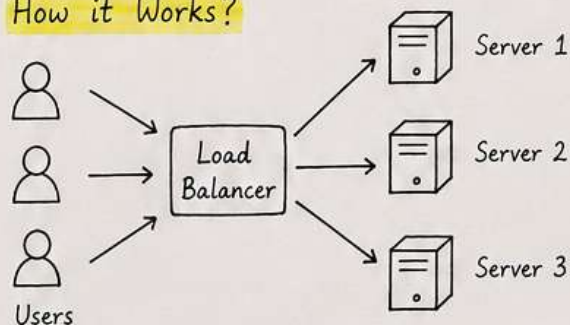
Key Takeaway

Design your system so that adding more resources is easy and doesn't require big changes in the architecture. ★

②. LOAD BALANCER

A Load Balancer distributes incoming traffic across multiple servers to ensure no single server is overwhelmed.

How it Works?



Common Algorithms

- Round Robin - Distribute requests one by one.
- Least Connections - Send to server with least active connections.
- IP Hash - Send same IP to same server.
- Weighted Round Robin - Distribute based on server weight.

Benefits

- High Availability
- Better Performance
- Fault Tolerance
- Easy Scalability

Key Takeaway

Load Balancer is the traffic cop of your system. It improves availability, reliability and performance. ★

A well balanced system is a fast, reliable and happy system! 😊

③ CACHING

Caching is the process of storing frequently accessed data in a fast storage layer so that future requests can be served quickly.

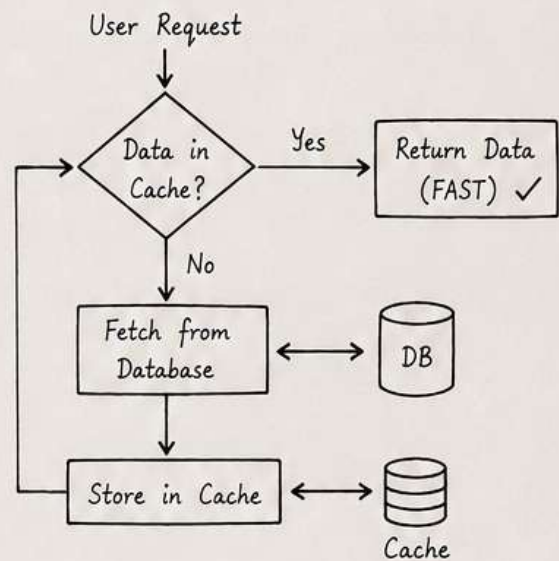
Types of Cache

- Browser Cache - Stores static assets (CSS, JS, Images) in browser.
- Application Cache - Stores data in application memory.
- Distributed Cache - Stores data in external systems like Redis or Memcached.

Benefits

- Reduces latency
- Decreases database load
- Improves system performance
- Saves bandwidth
- Improves user experience ★

How Caching Works?



Common Cache Strategies

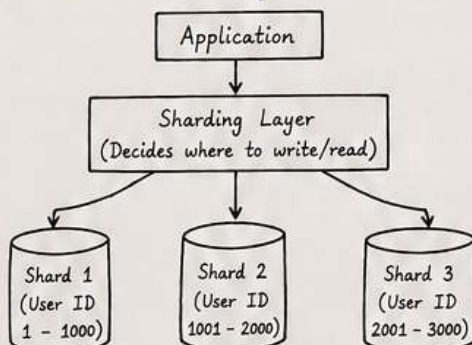
- Cache Aside - Application manages cache.
- Write Through - Write to cache and DB.
- Write Behind - Write to cache, DB updated later.
- LRU (Least Recently Used) - Evict least recently used data.
- TTL (Time To Live) - Expire data after certain time.

④ DATABASE SHARDING

Sharding is a technique to split a large database into smaller pieces called shards, each running on separate servers.

Why Sharding?

- Handle large amounts of data
- Improve read/write performance
- Distribute load across servers
- Scale horizontally



Data is split based on shard key (e.g., User ID). 😊

Types of Sharding

- Range Based - Based on range of shard key.
- Hash Based - Based on hash of shard key.
- Directory Based - Uses lookup table to map data.

Things to Consider

- Choosing the right shard key is important.
- Over-sharding can increase complexity.
- Re-sharding is needed as data grows.
- Cross-shard queries can be expensive. ★

Key Takeaway

Sharding helps in scaling databases horizontally but adds complexity in data management and queries. ★

5. REPLICATION

Replication is the process of copying data from one database to multiple databases to improve availability, reliability and fault tolerance.

Why Replication?

- High Availability
- Fault Tolerance
- Read Scalability
- Data Backup & Disaster Recovery

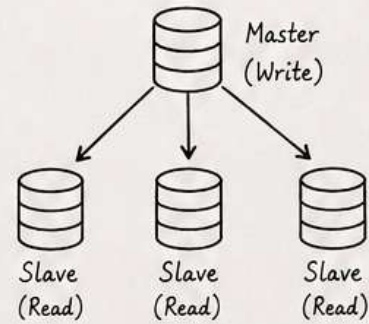
Types of Replication

- Master - Slave Replication
- Multi - Master Replication

Key Takeaway

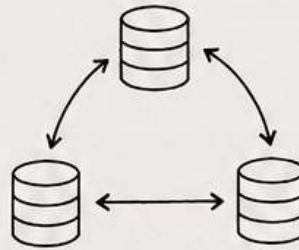
Replication keeps multiple copies of data so that if one server fails, others can still serve the data. ★

Master - Slave Replication



All write operations go to Master.
Read operations can go to any Slave.

Multi - Master Replication



All nodes can read and write.
Changes are replicated among all nodes.

6. KAFKA

Apache Kafka is a distributed event streaming platform used for building real-time data pipelines and streaming applications.

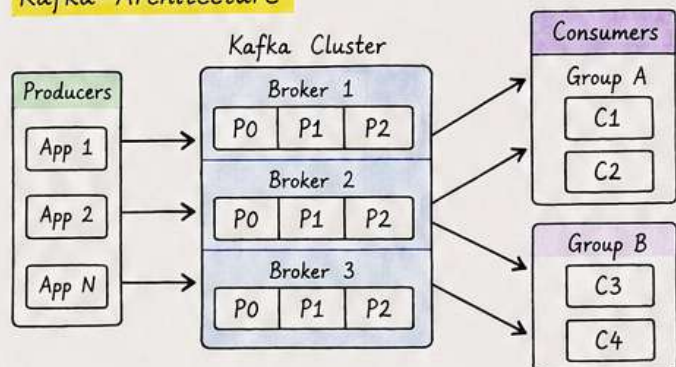
Key Concepts

- Producer - Publishes messages to a topic.
- Topic - A category/feed name to which messages are published.
- Partition - Topic is split into partitions for parallelism.
- Broker - Kafka server that stores data.
- Consumer - Reads messages from topic.
- Consumer Group - Group of consumers that work together.

Benefits

- High Throughput
- Scalable
- Fault Tolerant
- Durable
- Real-time Data Streaming ★

Kafka Architecture



Messages are distributed across partitions and brokers.
Consumers in the same group share the work.

Key Takeaway

Kafka is designed for high-throughput, fault-tolerant streaming of data. It decouples systems and allows real-time processing at scale. ★

7. REDIS

Redis is an in-memory data structure store used as a database, cache and message broker.

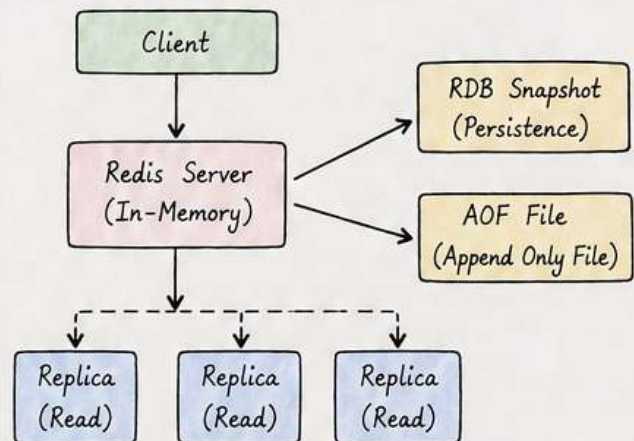
Key Features

- In-memory - Extremely fast
- Supports rich data structures
- Persistence - RDB, AOF
- High availability with replication
- Pub/Sub messaging
- TTL (Time to Live) for keys

Common Use Cases

- Caching
- Session Store
- Leaderboard, Counters
- Rate Limiting, Queues
- Real-time Analytics ☆

Redis Architecture



Data is written to the master.
Reads can be served by replicas.

Why Use Redis?

- Sub-millisecond response time
- Reduces load on primary database
- Handles high throughput
- Improves overall system performance ☆

8. API GATEWAY

API Gateway acts as a single entry point for all client requests and routes them to the appropriate microservices.

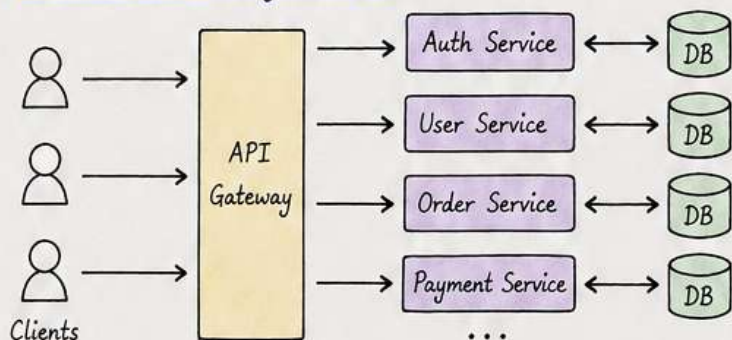
Responsibilities

- Request Routing
- Authentication & Authorization
- Rate Limiting
- Request/Response Transformation
- Logging & Monitoring
- Caching
- SSL Termination
- Protocol Translation

Benefits

- Simplifies client interactions
- Improves security
- Centralized control
- Scalability
- Better monitoring & analytics ☆

How API Gateway Works?



1. Client sends request to API Gateway.
2. Gateway processes request (auth, rate limit, etc.).
3. Gateway routes request to the correct microservice.

Key Takeaway

API Gateway is the front door of your system. It manages traffic, enforces policies and ensures secure, efficient communication between clients and services. ☆

9. RATE LIMITER

Rate Limiter controls the rate of requests to prevent abuse and ensure fair usage of resources.

Why Rate Limiting?

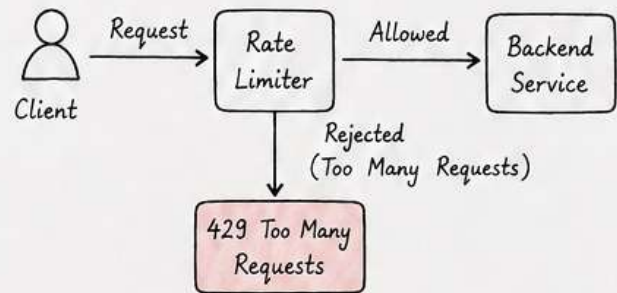
- Prevent abuse (DDoS, brute force)
- Ensure fair usage
- Protect backend services
- Avoid excessive costs
- Improve system stability

Common Algorithms

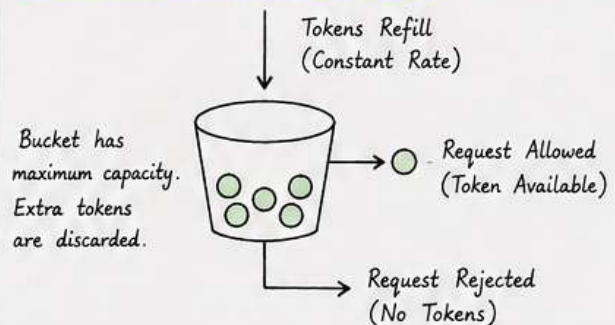
- Fixed Window Counter
- Sliding Window Log
- Sliding Window Counter
- Token Bucket
- Leaky Bucket



How Rate Limiter Works?



Token Bucket Algorithm (Example)



Key Takeaway

Rate limiting ensures system protection and fair usage of resources.

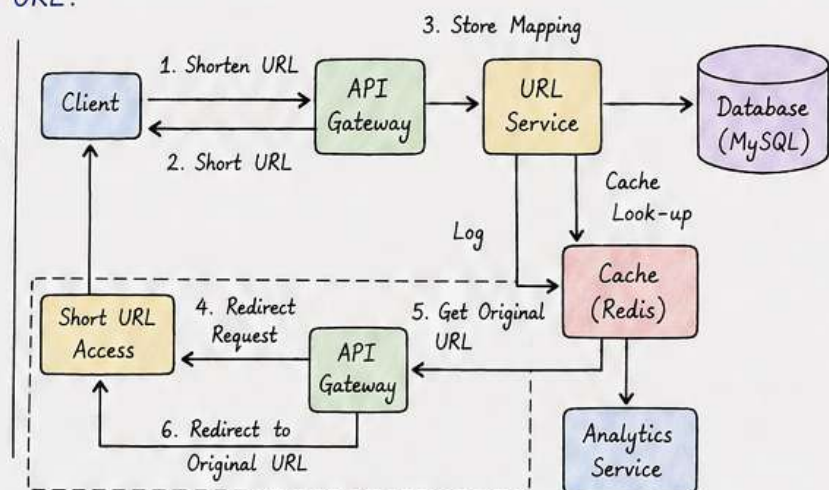


10. URL SHORTENER DESIGN

A URL Shortener converts long URLs into short, unique links that redirect to the original URL.

Requirements

- Generate short and unique URLs
- Redirect to original URL
- High availability & low latency
- Handle millions of requests
- Analytics (optional)



Key Components

- ① API Gateway - Handles all incoming requests.
- ② URL Service - Generates short URL and handles business logic.
- ③ Database - Stores mappings of short URL → original URL.
- ④ Cache (Redis) - Speeds up lookups for redirects.
- ⑤ Analytics Service - Tracks clicks and usage statistics.

Key Takeaway

A well-designed URL shortener is simple at the core but needs scalability, caching and analytics to handle massive traffic.

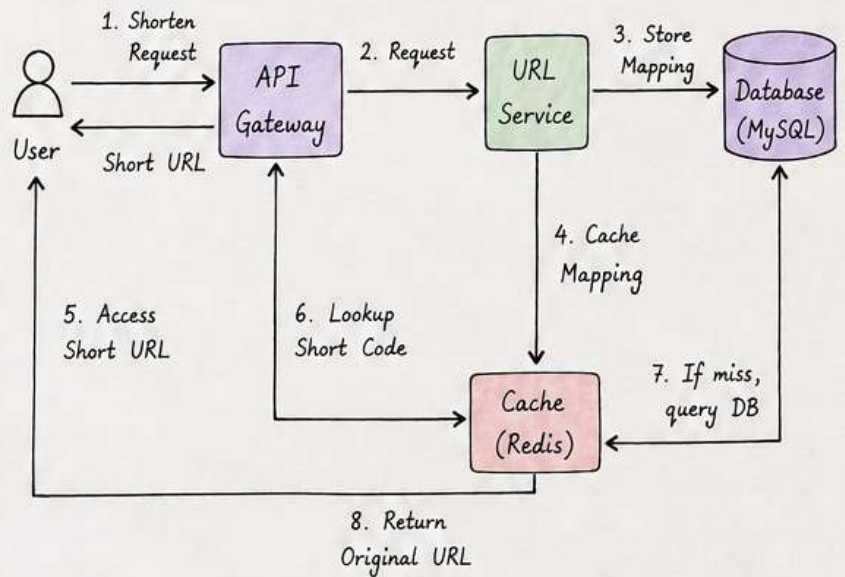


URL SHORTENER DESIGN (Contd.)

How it Works?

- ① User submits a long URL.
- ② URL Service generates a short code (e.g., abc123).
- ③ Mapping (short code → long URL) is stored in the database.
- ④ User gets the short URL.
- ⑤ When someone clicks the short URL, it is found in cache first.
- ⑥ If not in cache, query DB.
- ⑦ Redirect user to the original URL.

High Level Architecture



Short Code Generation

- Encode a numeric ID into Base62 (0-9, a-z, A-Z).
- Ensures short and unique codes.
- Example:
ID 100000 → "2or0" (Base62)

Scalability Considerations

- Use distributed systems for high availability.
- Shard the database based on short code range.
- Use CDN for static content.
- Cache hot data in Redis.
- Monitor & autoscale services.

Design Decisions

- Base62 encoding → Short & URL friendly.
- Cache first → Reduce DB load.
- Separate Read/Write DB → Better performance.
- Stateless services → Easy to scale.
- Asynchronous logging → Improve latency.

Optional Features

- Custom aliases (e.g., user defined short links)
- Link expiration
- QR code generation
- Click analytics & geo-tracking
- Bulk short URL generation

Key Takeaway

A good URL shortener handles massive scale with simple components, efficient encoding, caching and analytics. ★

Summary

These are the key building blocks in System Design.

Master them, mix them and design any scalable system! 😊

Keep Learning
Keep Building
Keep Scaling! ★

10. URL SHORTENER DESIGN (Contd.)

Data Modeling

- We need a mapping between short code and long URL.

id (PK)	short_code	long_url	created_at	clicks
1	abc123	https://example.com/a	2024-05-01	120
2	xyz789	https://example.com/b	2024-05-01	45
3	pqr456	https://example.com/c	2024-05-02	302
...	...	...	...	...

Short Code Generation Approaches

- Base62 Encoding of Auto Increment ID
e.g., 100000 → "2or0"
Simple and efficient.
 - Random String Generation
Generate random alphanumeric string and check uniqueness.
 - Hashing
Hash the long URL and encode to short string.
- Base62 is most commonly used!

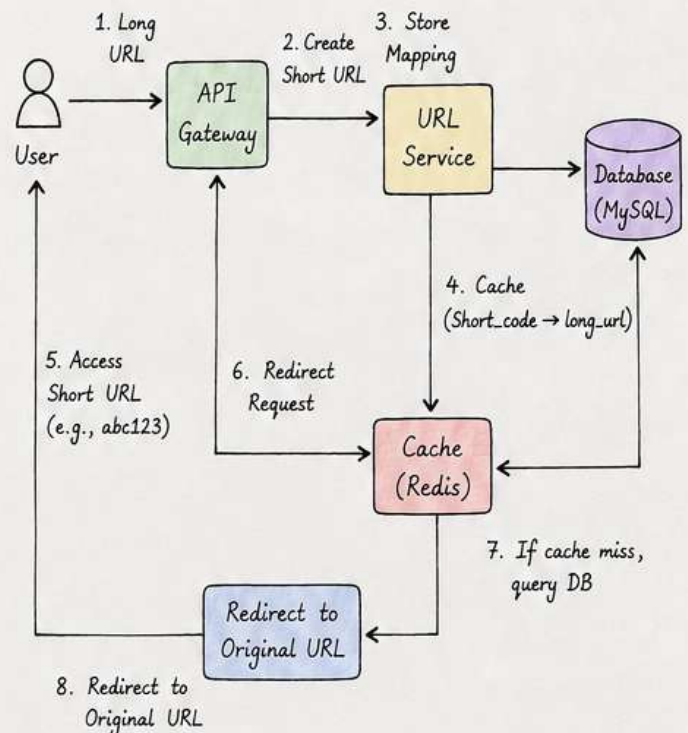
Base62 Encoding

- Uses 62 characters:
0-9, A-Z, a-z
- Example:
Decimal: 100000
 $\div 62 = 1612 \text{ R } 16 \text{ (G)}$
 $1612 \div 62 = 26 \text{ R } 0 \text{ (0)}$
 $26 \div 62 = 0 \text{ R } 26 \text{ (A)}$
Short code = AOG
- Short codes are compact and URL friendly.

Base62 Characters

0 - 9	(10)
A - Z	(26)
a - z	(26)
Total = 62	

High Level Architecture



Scalability Strategies

- Use Cache (Redis) to reduce DB load.
- Database sharding for storing mappings.
- Use CDN to serve static content.
- Asynchronous logging for click analytics.
- Rate limiting to prevent abuse.

Metrics to Track

- Total URLs created
- Total clicks
- Top performing links
- Error rate & latency
- Cache hit ratio



Edge Cases

- Same long URL multiple times → return existing short URL.
- Invalid or expired short code.
- Malicious users generating too many URLs.
- Very high traffic for a particular short code.
- Database failures.



Key Takeaway

A well designed URL shortener is simple on the surface but requires careful consideration of encoding, caching, scalability, and analytics to handle massive scale efficiently.



Summary URL Shortener is a great example of how small features require deep system design thinking to build something scalable and reliable. 😊

Tiny link today, huge impact tomorrow! 😊

10. URL SHORTENER DESIGN (Contd.)

Workflow Summary

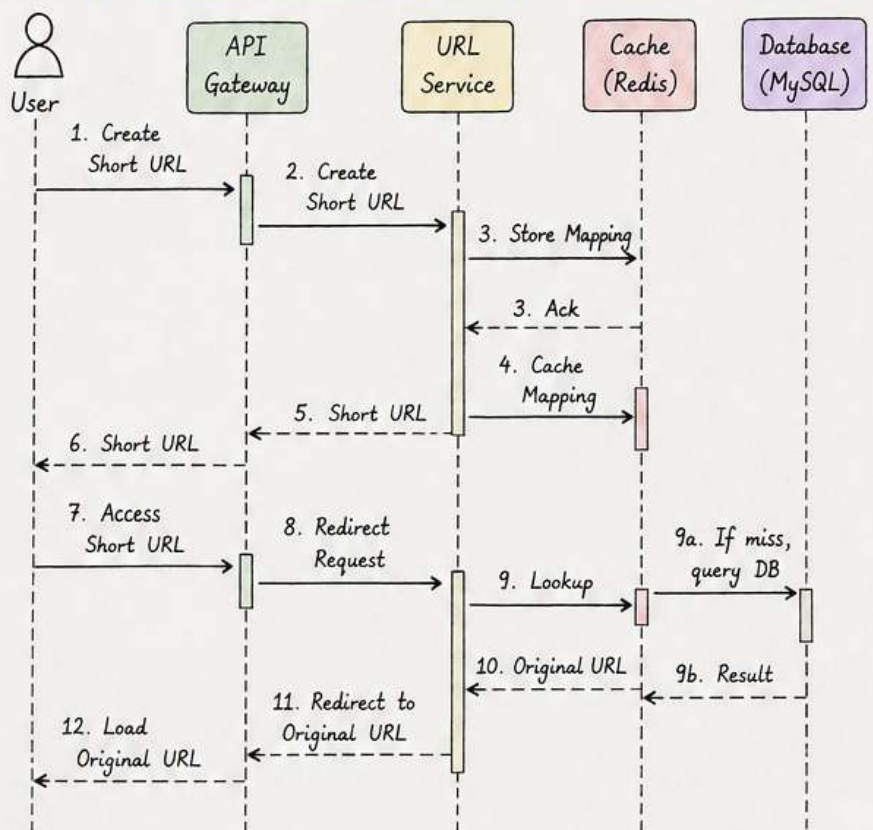
- ① User submits a long URL.
- ② API Gateway forwards request to URL Service.
- ③ URL Service generates short code and stores mapping in DB.
- ④ Short URL is returned to user.
- ⑤ When short URL is accessed, redirect user to original URL.

Example Flow

- Long URL : <https://example.com/very/long/path/to/resource>
- Short Code : abcd123
- Short URL : <https://sho.rt/abcd123>
- Redirects to the original long URL.



Sequence Diagram (High Level)



Database Schema (Example)

Column Name	Type	Description
id (PK)	BIGINT	Unique ID
short_code	VARCHAR(16)	Short code (unique)
long_url	VARCHAR(2083)	Original long URL
created_at	DATETIME	Creation timestamp
clicks	BIGINT	Total clicks on link

- Add index on short_code for fast lookup.

Scaling Considerations

- Use load balancer to distribute traffic.
- Stateless services for horizontal scaling.
- Use Redis Cluster for high availability.
- Shard database based on short_code range.
- Use CDN for static assets and landing pages.
- Monitor metrics and autoscale as needed.



Short Code Length vs Capacity

Length	Possible Combinations	Suitable For
5	$62^5 \approx 916\text{M}$	Small scale
6	$62^6 \approx 56\text{B}$	Medium scale
7	$62^7 \approx 3.5\text{T}$	Large scale
8	$62^8 \approx 218\text{T}$	Very large scale

- Using Base62 gives best trade-off between length and capacity.



Additional Features (Optional)

- Custom Alias
- Expiry Time (TTL)
- Password Protection
- Click Analytics
- QR Code Generation
- Bulk URL Shortening
- Branded Domains
- UTM Parameter Support
- Link Preview

Key Takeaway

A URL shortener may look simple, but handling scale, performance, and reliability at billions of requests per day requires thoughtful system design, the right technologies, and constant monitoring.

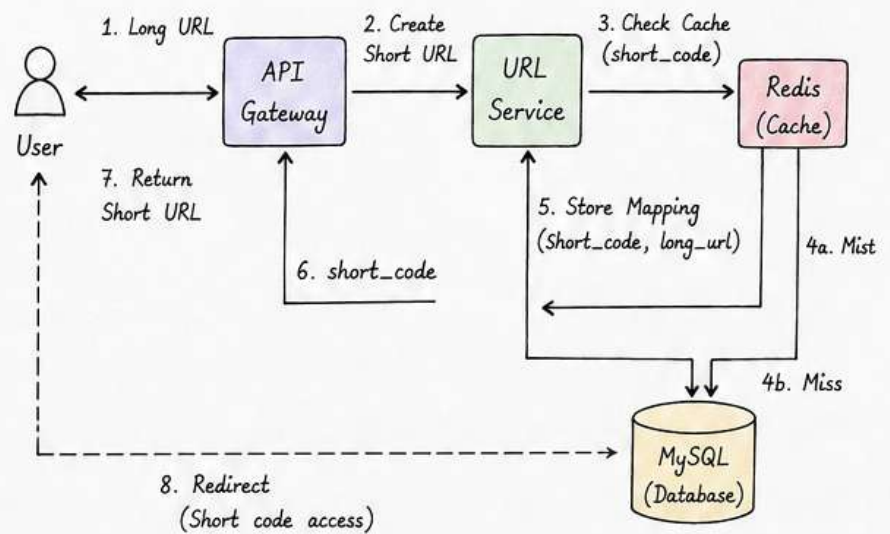


10. URL SHORTENER DESIGN (Contd.)

Component Responsibilities

- API Gateway : Routes requests, handles rate limiting, auth.
- URL Service : Generates short code, handles business logic.
- Cache (Redis) : Fast lookup for short code → long URL.
- Database (MySQL) : Stores persistent mappings.
- User (Client) : Creates and accesses short URLs.

Low Level Design (Core Flow)



Redirection Flow

- ① User clicks short URL.
- ② API Gateway forwards the request.
- ③ URL Service checks Redis cache.
- ④ If found → return long URL.
- ⑤ If not found → query DB.
- ⑥ Cache the result in Redis.
- ⑦ Redirect user to long URL.

Example Walkthrough

User submits : <https://example.com/blog/how-to-scale>

Generated Short Code : abc123

- Stored in DB → (abc123, <https://example.com/blog/how-to-scale>)
- Next time someone accesses <https://sho.rt/abc123>
 - Check Redis (abc123)
 - If miss, check DB
 - Redirect to original long URL

Caching Strategy

- Cache Key : short_code
- Cache Value : long_url
- TTL : e.g., 7 days (configurable)
- Eviction Policy : LRU
- Benefit : Reduces DB load, improves latency ★

Database Indexing

- Primary Key on id
- Unique Index on short_code
- Index on created_at (for analytics/cleanup)
- Partitioning (optional) on created_at for large scale ★

Handling Collisions

- Use Base62 encoding with auto-increment ID.
- If collision happens (rare), generate next ID.
- Ensure unique constraint on short_code in DB.

Example ID 1000000 → Base62 → "2or0"
If "2or0" exists, try ID 1000001 → "2or1"
Continue until unique. ★

Reliability & Availability



- Redis Cluster for high availability
- Multi-AZ deployment
- CDN for static assets
- Health checks & auto failover
- Regular backups & monitoring ★

Key Takeaway

A URL Shortener is a classic system design problem that touches almost all core concepts – scalability, caching, databases, load balancing, and reliability. Simple on the surface, but requires careful planning to handle billions of requests. ★

10 URL SHORTENER DESIGN (Contd.)

Putting It All Together

- User submits long URL.
- System generates short code.
- Mapping is stored in DB.
- Short URL is returned.
- On access, system looks up mapping in cache first, then DB.
- If found → redirect to original URL.
- If not found → 404 / invalid.



Error Handling

- Invalid short code → 404 Not Found
- Expired link → 410 Gone
- Rate limit exceeded → 429 Too Many Requests
- Database down → Use cache + retries
- Service down → Return friendly message



Capacity Estimation (Example)

New URLs per day	10 Million
Read : Write ratio	100 : 1
Total URLs	10M per day
Storage per mapping	~ 100 bytes
DB Storage per day	~ 1 GB
Cache size	~ 10 GB
Expected QPS (reads)	~ 10K - 50K

- Scale horizontally at each layer.



Security Considerations

- Validate and sanitize input.
- Rate limit by IP / user.
- Prevent brute force guessing.
- Use HTTPS everywhere.
- Short codes should be hard to predict.

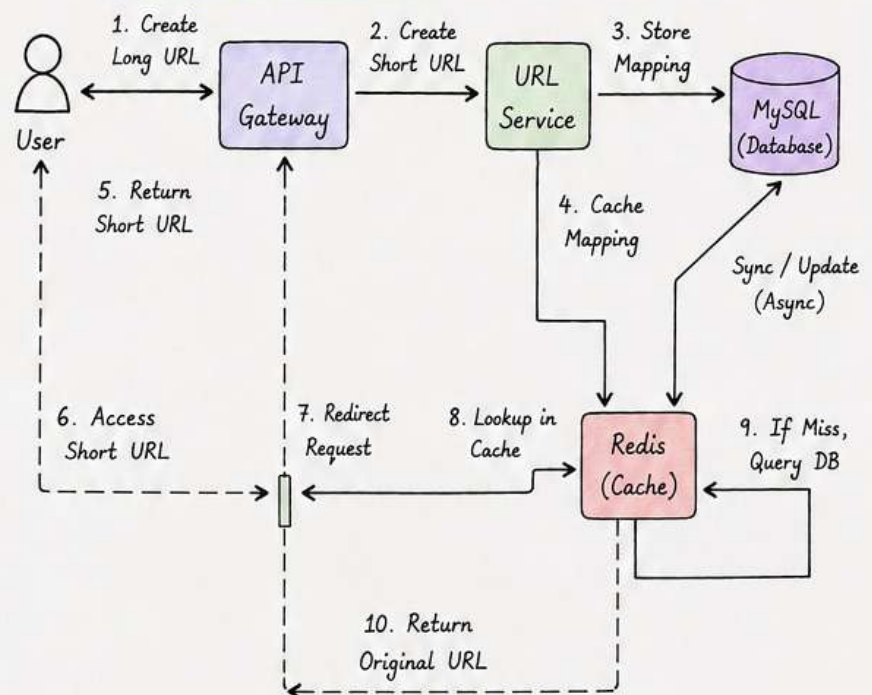


Final Key Takeaway

A URL Shortener may look simple, but building one at scale is a great exercise in system design. Focus on the core requirements first, then scale each component with the right trade-offs. Keep it simple, observable and reliable.



Complete High Level Flow



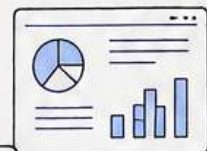
Tech Stack (Example)

- Frontend : Web / Mobile
- API Gateway : Nginx / Kong / AWS API Gateway
- Backend : Go / Java / Node.js
- Database : MySQL / PostgreSQL
- Cache : Redis Cluster
- Load Balancer : HAProxy / Nginx
- CDN : CloudFront / Fastly
- Monitoring : Prometheus + Grafana
- Logging : ELK Stack



Possible Enhancements

- Custom short codes (user defined).
- Link analytics (clicks, location, device).
- Expiration / Active time window.
- Password protected links.
- QR code generation.
- Link preview before redirect.



Small Link
Big Impact



Keep Building, Keep Learning, Keep Scaling! 😊

♥ Design • ♥ Build • ♥ Scale • ♥ Repeat • 😊

10 URL SHORTENER DESIGN (Contd.)

Page 13

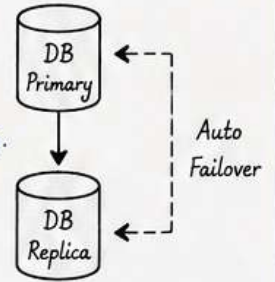
Database Operations

- Insert Mapping
Store (short_code, long_url, created_at)
- Lookup by short_code
`SELECT long_url FROM urls
WHERE short_code = ?`
- Analytics Queries
`SELECT * FROM urls
WHERE created_at BETWEEN
? AND ?`



Data Consistency & Reliability

- Use transactions for writes.
- Replicate DB for high availability.
- Regular backups & point-in-time recovery.
- Handle failures gracefully with retries.
- Monitor system health & alerts.



API Endpoints (Example)

- ① POST /api/shorten
Body: { long_url }
Response: { short_url, short_code }
- ② GET /api/{short_code}
Response: Redirect to long_url
- ③ GET /api/stats/{short_code}
Response: { clicks, created_at, ... }
- ④ DELETE /api/{short_code} (optional)
Response: { success: true }



Monitoring Metrics

- ✓ Requests per second (RPS)
- ✓ Error rate (%)
- ✓ Average latency (ms)
- ✓ Cache hit ratio (%)
- ✓ DB CPU, Connections
- ✓ Storage used
- ✓ Top short codes by clicks



Observe → Analyze → Improve



Trade-offs

Choice	Trade-off
Base62 Encoding	Short & compact IDs, but not sequential
Caching	Improves read performance, extra complexity
Database Sharding	Highly scalable, more complex to implement
Pre-generation vs On-demand codes	Pre-gen is fast, but wastes space if not used
Storing analytics	Detailed analytics needs more storage & processing
CDN for redirects	Reduces latency, but adds cost

Design is simple,
Scale is hard,
Observability is everything!



Real World Examples

- bit.ly
- tinyurl.com
- t.co (Twitter)
- shorturl.at
- goo.gl (legacy)



Things to Discuss in Interview

- ① Requirements & constraints
- ② High level architecture
- ③ Core components deep dive
- ④ Data modeling
- ⑤ Scalability & optimizations
- ⑥ Monitoring, security & trade-offs

Always ask questions & clarify requirements first! 😊

Summary

URL Shortener is a classic system design problem that covers many important topics - APIs, caching, databases, encoding, scalability, reliability and monitoring. Master the building blocks and think about scale & trade-offs! 😊



Learn Today, Build Tomorrow, Scale Forever! 😊

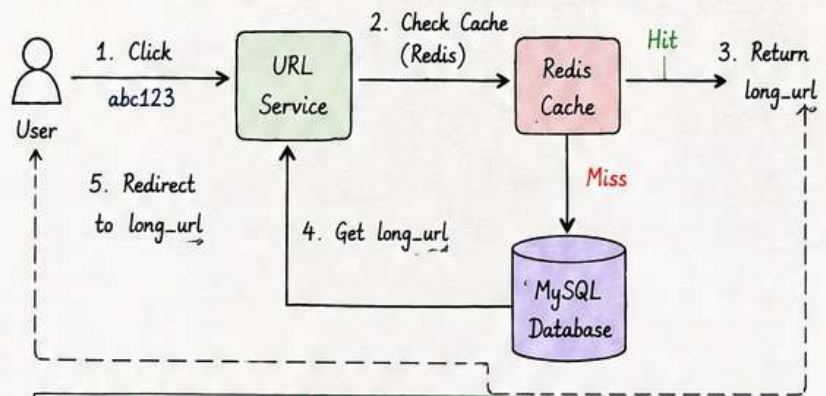
♥ Design • ♥ Build • ♥ Scale • ♥ Repeat • 😊

Advanced Considerations

- Custom short codes (user defined)
- Link expiration (TTL)
- Dynamic links (update destination)
- Link alias / branded domains
- Click analytics (by location, device, referrer)
- A/B testing for links
- Password protected links
- Deep linking for mobile apps
- Bulk URL shortening (batch)



Read Path in Detail



Always try cache first.
Fallback to DB only if cache miss.

ID Generation Strategies

- ① Auto Increment ID → Simple but sequential
- ② UUID → Large but globally unique
- ③ Snowflake ID → Unique + time sortable
- ④ Hash of URL → Not good (collisions)
- ⑤ Random + Check → Good balance (popular)



Caching Example

- Key : short_code (e.g. abc123)
- Value : long_url
- TTL : 7 days (configurable)
- Eviction : LRU (Least Recently Used)

Cache makes huge difference!

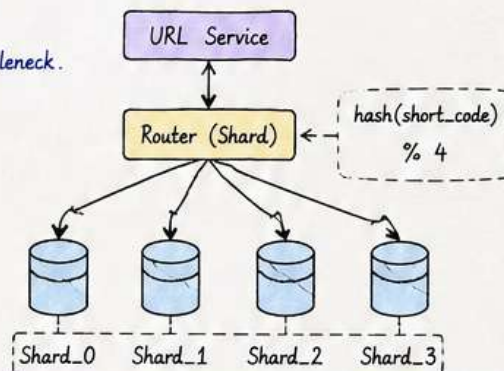


Database Sharding (Scale Write)

- When data grows, single DB can be a bottleneck.
- Shard based on short_code hash.
- Example: 4 shards

$\text{shard_id} = \text{hash}(\text{short_code}) \% 4$
route to shard_{shard_id}

- Each shard stores subset of data.
- Use consistent hashing for easy scaling.



Benefit

- ✓ High write throughput
- ✓ Horizontal scaling
- ✓ Fault isolation
- ✓ Easy to add more shards

Rate Limiting

- Prevent abuse & brute force.
- Limit creation requests per IP / user.
- Use Redis Token Bucket / Leaky Bucket.

Example: 100 requests per minute per IP



Security Best Practices

- Validate all inputs (long URL, custom code).
- Prevent open redirect attacks.
- Use HTTPS everywhere.
- Rate limit & monitor suspicious activity.
- Sanitize & encode before redirecting.



Cost Optimization

- Use CDN for static assets.
- Aggressive caching.
- Compress responses.
- Monitor & cleanup expired links.



Common Pitfalls

- ⊗ Not handling collisions in short codes
- ⊗ DB becomes bottleneck (no sharding)
- ⊗ Not using cache
- ⊗ No monitoring & alerts
- ⊗ Poor analytics / no tracking



Key Takeaway

A URL Shortener is a perfect example of how a tiny feature requires deep system thinking. It touches scaling, caching, databases, security, analytics and much more. Keep it simple, build it right, and scale step by step!



10. URL SHORTENER DESIGN (Contd.)

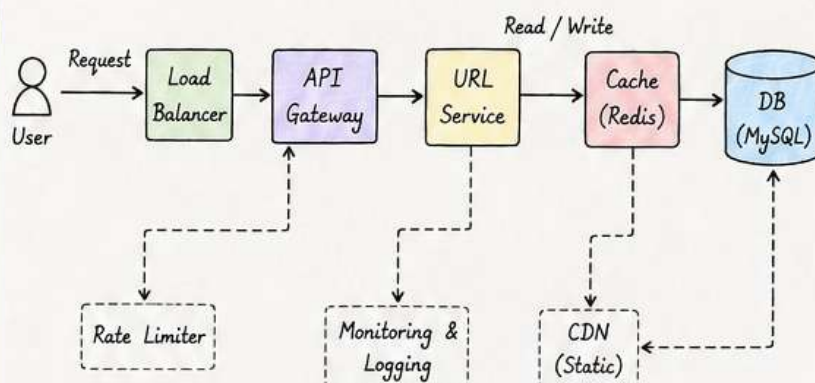
Page 15

End to End Flow (Summary)

- 1 User submits long URL.
- 2 API Gateway validates & forwards.
- 3 URL Service generates short code.
- 4 Mapping (short_code → long_url) stored in DB.
- 5 Response returned to user.
- 6 On access, short code is looked up.
- 7 If found → redirect to long URL.
- 8 If not found → 404 / error.



System Architecture (High Level)



Keep services stateless, cache aggressively, distribute load & monitor continuously.

Short Code Generation

- Use Base62 (0-9, a-z, A-Z)
- Encode auto-increment ID to Base62.
- Length 6 → 56B combinations
- Length 7 → 3.5T combinations
- Ensure uniqueness (handle collision).
- Custom codes (optional) handled separately.



Redirection Handling

- Check cache first (Redis).
- If miss → query DB.
- If found → redirect (301/302).
- If not found → 404.
- Cache the result for faster subsequent requests.



Scalability Tips

- ✓ Stateless services
- ✓ Horizontal scaling
- ✓ Redis cluster
- ✓ DB sharding
- ✓ CDN for static content
- ✓ Async logging & metrics



Example Walkthrough

- User enters : `https://example.com/very/long/path`
- Short code generated : `xs9A1b`
- Stored in DB : (`xs9A1b` → `https://example.com/very/long/path`)
- User shares : `https://sho.rt/xs9A1b`
- Someone clicks :
 - Check Redis (hit/miss)
 - Get long URL from DB
 - Redirect to original URL



Monitoring & Analytics



Track clicks, uniques, referrers.



Monitor latency, error rate.



Alerts for failures, high load.



Top links, locations, devices.



Key Components Recap

- **API Gateway** : Entry point, auth, rate limiting.
- **URL Service** : Core logic, short code generation.
- **Cache (Redis)** : Fast lookup for mappings.
- **Database** : Persistent storage of mappings.
- **CDN** : Serve static assets, reduce latency.
- **Load Balancer** : Distribute traffic across servers.
- **Monitoring** : Observability & alerting.



Final Checklist

- ☒ Short code generation strategy
- ☒ Unique & collision handling
- ☒ Cache strategy (key, TTL, eviction)
- ☒ DB schema & indexing
- ☒ Redirection logic & error handling
- ☒ Rate limiting & security
- ☒ Monitoring & logging in place
- ☒ Scalability & high availability



Final Takeaway

A URL Shortener is small on the surface but a great example of system design. It teaches us how to balance simplicity, scalability, performance, and reliability using the right technologies and trade-offs.

